

# A cleaner null model for site-determining ion matches

Sequence decoys vs a within-target regression: options and the recommended framing

Pioneer.jl / phospho-localization work

2026-07-09

## The problem this note addresses

The exploration in `matched_ions_empirical_exploration.pdf` established the distribution of matched site-determining ions for competing phospho-localizations and made the point that the “competitor rate” reported there is a mixture:

- (a) coincidental matches from spectrum density (the noise floor an AScore-analog is trying to model), plus
- (b) real matches from co-eluting positional isomers, which the same isolation window co-fragments.

Because (b) is nonzero and cannot be separated from (a) without extra information, the observed competitor rate is only an *upper bound* on the true wrong-localization null  $p_0$ . To build a defensible probability model for localization confidence we need  $p_0$  estimated from data where (b) is either absent by construction or explicitly modeled.

This note compares two candidate strategies and recommends the second.

## Option A: sequence decoys as the null

### The idea

Pioneer’s search already carries a full sequence-decoy library (reversed/randomized protein sequences at the target’s charge). A sequence decoy is *not* in the sample, so its predicted b/y ions should have no genuine fragmentation support: observed matches, if any, are pure spectrum density  $\times$  mass-tolerance chance. If we simulate site-determining ions on decoys the same way we do on targets, the observed decoy match rate is a candidate estimate of  $p_0$ .

### Why it looks appealing

- Requires no new modeling code — the search already yields decoy PSMs and their associated scans.
- Chemistry-matched to the target space (same mass, similar amino-acid composition on average), so decoy fragments cover the same fragment m/z range as targets.

## The bias hidden in “score decoys on their own scans”

A decoy PSM has a scan because the search’s scoring assigned it a scan — which means the decoy happened to accumulate *some* accidental support at that scan. Decoy scans are therefore *conditioned on non-zero accidental support*: they are enriched for scans where the decoy’s b/y m/z lines happen to line up with real peaks. Scoring decoy site-determining ions on that scan will produce a match rate systematically *higher* than pure chance density on random scans.

Concretely: pick a random decoy sequence and a random MS/MS scan, and its b/y lines match observed peaks at rate  $p_{\text{chance}}$  (a per-scan density number). But among the decoys the search *reported*, the scan choice is correlated with unusually high accidental match — that biases the estimate upward.

## Fixes for that bias, and their new problems

1. **Score decoy fragments on a random scan** (or a mass-matched scan chosen independently of the decoy). This breaks the conditioning bias but discards the scan-specific density information — the resulting  $p_0$  is a dataset-average, not a per-scan number, so it can’t represent the  $\sim 7\times$  range in null match rates we saw between sparse and dense spectra.
2. **Score decoy fragments on the target’s scan** (mass-matched). This restores the per-scan density information but requires a mass shift that changes the fragment m/z landscape, and the decoy’s b/y positions are now unrelated to which peaks are present — we’re back to something equivalent to a pure density estimate, which we can compute directly without invoking decoys at all (see Option B, `p_chance` below).

## The other concern: decoys generally match less overall

A separate reason to be cautious with sequence decoys as the null is that a decoy’s *overall* b/y ion match rate is much lower than a real target’s — decoy sequences don’t correspond to real fragmentation. If we condition on decoys that pass any q-value threshold, we’re selecting decoys that got lucky; if we don’t condition, we’re mixing “search-quality” decoy PSMs with “no evidence” ones. Neither is a clean population.

## Verdict on Option A

Sequence decoys are a useful *calibration probe* — a way to check that a null model we build elsewhere behaves as expected out-of-sample — but they are a shaky basis for *constructing* the null. We recommend using them for validation only.

## Option B: within-target regression + per-scan density null

### The idea

Fit two functions on data we already have:

- $p_{\text{real}}(\text{features}) = \Pr(\text{match} \mid \text{ion is really in the scan and Prosit predicts it})$ , learned from *all* b/y ions of all confident (target-passing) PSMs.
- $p_{\text{chance}}(\text{scan}, m/z) = \Pr(\text{match} \mid m/z \text{ is random noise})$ , computed per scan from the empirical peak density.

Then for each site-determining ion of each pair we evaluate both, and combine into a per-pair log-likelihood ratio between “target is right, competitor is wrong” and its reverse.

### **p\_real: the “if this ion is a real fragment, does it match?” model**

Fit a logistic regression (or LGBM if you want interactions for free) on all (PSM, ion) rows from confident target PSMs.

- **Label.**  $\mathbb{1}[\text{ion m/z matches an observed peak within 20 ppm}]$ .
- **Features.**
  - Prosit predicted relative intensity (log-scaled)
  - Ion type (b vs y)
  - Ion position (normalized,  $i/(L - 1)$ )
  - Peptide length  $L$
  - Fragment charge  $z$
  - Scan peak count (log)
  - Deconvolution weight for the precursor at this scan (proxy for “how much of this precursor is really there”)
  - Fragment m/z (mass-dependent detector effects, coarse)
  - Number of observed peaks *inside*  $\pm 20$  ppm around expected m/z (local density around the ion in question)
  - Precursor intensity / TIC (spectrum-level abundance handle)
- **Output.** A calibrated match probability per ion, per PSM.

Rows: 969k passing PSMs  $\times$   $\sim 30$  ion candidates each  $\sim 30$ M rows. Fast to fit with LGBM. The regression is a *positive-control* model of fragmentation — it captures the joint effect of Prosit intensity, spectrum noise, and precursor abundance on the probability of seeing a real fragment.

### **p\_chance: the density-only null**

Per scan, the null probability that a random m/z in the fragment range matches any observed peak within  $\pm 20$  ppm:

$$p_{\text{chance}}(\text{scan}, m/z) = \frac{\text{fraction of the fragment m/z axis covered by } \pm 20 \text{ ppm windows around observed peaks}}{1}.$$

Two easy estimators:

- **Analytical.** For each observed peak at  $m_k$ , its window is  $2 \cdot m_k \cdot 20 \cdot 10^{-6}$ . Sum window widths (unioned if any overlap) and divide by the total fragment m/z range considered (say 100 – 2000 m/z). One number per scan.
- **Empirical.** Draw 10,000 uniform m/z in [100, 2000], count matches, divide. Same result to two digits; useful as a sanity check.

The analytical form makes explicit that  $p_{\text{chance}}$  is *proportional* to peak count for sparse scans and *saturates* below one as scans get dense — which is exactly the shape the empirical stratification in the prior PDF already suggested (competitor rate 3% at 0–500 peaks, 20% at 3000+ peaks).

**p\_chance** is directly what AScore’s  $i/100$  is trying to approximate, but (i) per scan rather than global, (ii) using *all* peaks rather than the top  $i$ , and (iii) analytically defined rather than derived from an ad-hoc top- $i$  window.

## Per-pair localization score (log-likelihood ratio)

For each pair  $(t, c)$  and each site-determining ion index  $j \in \text{sites}(t, c)$ :

- Compute predicted m/z under target’s phospho positions:  $m_j^t$ . Compute match indicator  $x_j^t = \mathbb{1}[\exists \text{ peak within } \pm 20 \text{ ppm of } m_j^t]$ .
- Compute predicted m/z under competitor’s positions:  $m_j^c$ , and  $x_j^c$  analogously.

Define two hypotheses:

- $H_t$ : target localization is the truth. Under  $H_t$ , target-side ions are drawn from  $\text{Bernoulli}(p_{\text{real},j}^t)$ , competitor-side ions from  $\text{Bernoulli}(p_{\text{chance},j}^c)$ .
- $H_c$ : competitor localization is the truth. Roles swap.

Per-pair log-likelihood ratio:

$$\text{LLR}(t \succ c) = \sum_j \log \frac{\Pr(x_j^t | H_t) \Pr(x_j^c | H_t)}{\Pr(x_j^t | H_c) \Pr(x_j^c | H_c)}.$$

Localization confidence for the target = softmax over  $\text{LLR}(t \succ c)$  across all competitors. A per-PSM score reported as  $-10 \log_{10}$  of the ratio between the top two candidates recovers an AScore-style number, but with a defensible per-scan, per-ion probability foundation.

## Why this handles the co-elution mixture correctly

If target and competitor genuinely co-elute, then both target-side and competitor-side ions are *really present* in the scan. Under  $H_t$ , target-side ions get  $p_{\text{real}}$  (high) but competitor-side ions were assumed to follow  $p_{\text{chance}}$  (low). The observed competitor-side match count is much higher than  $p_{\text{chance}}$  would predict, so the log-likelihood  $\Pr(x_j^c | H_t)$  term is small —  $H_t$  can’t explain the competitor’s observed matches. Under  $H_c$  the same thing happens in reverse. The LLR converges toward zero, correctly signalling “ambiguous — both localizations have evidence.” This is the *right* behavior: in a true co-elution scan we should not confidently pick one over the other.

By contrast, an AScore-analog that uses a single  $p_0$  (whether estimated from target-vs-competitor rates or from decoys) will systematically over-report confidence in co-elution scans, because it interprets the competitor’s real matches as chance.

## Handling the “competitor m/z has no Prosit intensity” issue

Under  $H_c$  we need  $p_{\text{real}}$  for the competitor’s shifted m/z lines, which means Prosit predictions for the *competitor* peptideform. Two options:

1. **Predict competitor library rows.** Prosit predicts all target isomers in the clean library already — for the isomers we already built, we can look up the competitor’s predicted intensities directly. This is essentially free.
2. **Reuse the target’s Prosit intensity at the same  $(b/y, i, z)$  as a proxy.** Predicted intensity for a b/y ion is more sensitive to sequence than to phospho position (phospho shifts m/z but rarely changes the ion’s ionization efficiency dramatically). Usually within a factor of 2. This is a shortcut for a v0 prototype; option (1) is the right thing for a production score.

## Where sequence decoys still play a useful role

Even in Option B they’re not wasted:

- **Out-of-sample calibration of  $p_{\text{chance}}$ .** Score decoy site-determining ions (on decoy PSMs’ scans — the conditioning bias is fine here because we’re testing, not fitting). The observed decoy match rate on site-determining ions should track  $p_{\text{chance}}$ . If it’s systematically above (say  $1.5\times$ ), we have a bias to correct; if below, we’ve conservative.
- **Empirical FDR of the LLR score.** The LLR is naturally testable against sequence decoys: rank all PSMs (targets + decoys) by their top-competitor LLR and compute decoy-hit rate at each threshold. This is standard search FDR estimation, applied to the *localization* score rather than the identification score.

## Practical tradeoffs

| Question                                                    | Option A<br>(decoy-null)      | Option B (regression +<br>density)            |
|-------------------------------------------------------------|-------------------------------|-----------------------------------------------|
| Handles co-elution mixture correctly?                       | No                            | Yes                                           |
| Per-scan density-adaptive?                                  | No (unless refit<br>per scan) | Yes                                           |
| Requires new modeling code?                                 | Minimal                       | Modest — one<br>logistic/LGBM fit             |
| Requires Prosit predictions for competitor<br>peptidoforms? | No                            | Yes (or a factor-of-2<br>proxy)               |
| Sensitive to conditioning bias in decoy selection?          | Yes                           | No                                            |
| Reuses the sequence-decoy set for calibration?              | It IS the null                | Yes, as validation                            |
| Gives an AScore-analog score end-to-end?                    | Yes (with caveats)            | Yes (LLR $\rightarrow -10\log_{10}$<br>ratio) |
| Answers “what’s the noise floor” cleanly?                   | No (biased)                   | Yes ( $p_{\text{chance}}$ )                   |

## Recommended path forward

1. **Compute  $p_{\text{chance}}$  per scan on the existing exploration Arrow** (5 minutes, no new modeling). This directly gives us the AScore-analog uniform- $p$  number *per scan*, and lets us split the observed competitor-rate into two parts:
  - Pairs where observed competitor rate  $\approx p_{\text{chance}} \rightarrow$  clean null; competitor is not co-eluting; density-only explanation works.
  - Pairs where observed competitor rate  $\gg p_{\text{chance}} \rightarrow$  excess signal is real co-elution.

The split is diagnostic. It tells us how much of the mixture (a) vs (b) is which, per stratum, without any regression.
2. **Fit  $p_{\text{real}}$  regression on the confident-target b/y ion set.** LGBM with the feature list above; 30M rows. Predict per-ion match probability; check calibration in bins. Confirm decoy site-determining match rate tracks  $p_{\text{chance}}$  (not  $p_{\text{real}}$ ).
3. **Compute LLR score per pair**, initially with the factor-of-2 Prosit proxy for competitor intensity to move fast.

4. **Estimate localization FDR** on the LLR score by sequence-decoy hit rate; compare to the isomer-decoy FLR the prior work has been using. The two FDR numbers should tell related stories.
5. **Iterate** — replace the intensity proxy with actual Prosit predictions for competitor isomers if the score is still leaving IDs on the table.

Step (1) is what I'd run first — it's a definitive test that gives us either a green light on Option B or a specific problem to fix, at essentially zero cost.

## Files (proposed)

- **p\_chance calculator + splitter** — `dev_docs/prosit_ptm_integration/prototype/split_null_by_pch` (new). Reads `explore_matched_counts_v1.arrow`, computes `p_chance` per scan from the `msdata` arrows, writes a per-pair Arrow with `p_chance` and `excess_over_chance` columns, prints stratified summaries.
- **p\_real regression trainer + scorer** — `dev_docs/prosit_ptm_integration/prototype/fit_p_real.jl` (new). Trains an LGBM  $P(\text{match} \mid \text{features})$  on all confident-target b/y ions; writes model and per-ion predictions.
- **LLR per-pair scorer** — `dev_docs/prosit_ptm_integration/prototype/score_llr.jl` (new). Reads the above and emits per-pair LLR, plus a per-PSM AScore-analog.
- **Related prior work.**
  - `matched_ions_empirical_exploration.md/.pdf` — the exploration that established the mixture problem.
  - `prototype/extract_sds_features.jl` — the S-ratio prototype.
  - `prototype/PROTOTYPE_STATUS.md` — S-score validation status.